

Stuff vs Map-Reduce vs Refine, Costed by Hand

Three control-flow shapes for “process N inputs, produce one output” — every token, every second, and every overflow counted for one batch of reviews.

What this document does. It takes the one tiny workload from Module 2.2 — the *review-analysis* fan-out that turns a list of product reviews into a single executive summary — fixes the size (in tokens) of every input, and then computes the token cost, the wall-clock latency, and the overflow point for all three strategies: STUFF, MAP-REDUCE, and REFINE. Nothing is hand-waved. We use $D = 20$ reviews of $d = 800$ tokens each, prompt overhead $B = 500$, a context window $W = 8000$, and a map summary of $s = 100$ tokens. Every figure below is reproduced by the small Python program in Appendix A, so the arithmetic is exact and self-consistent.

Where this sits. This is **Topic 1 of Module 2.2** — the costing behind the module’s “Decision Flow” (*fits in context* $\rightarrow$ *Stuff*; *independent items* $\rightarrow$ *Map-Reduce*; *order matters* $\rightarrow$ *Refine*). Its companions: Module 2.2 Topic 2 (hierarchical reduce — what to do when even the map-reduce reduce step overflows) and Module 0.0 (the ReAct cost curve — the *same* token accounting, applied to a serial loop instead of a parallel fan-out).

Concepts covered, in order: the three strategies as token accounting · prompt overhead B vs. payload $D \cdot d$ · STUFF’s single call and its hard window ceiling · MAP-REDUCE’s D map calls plus one reduce · why parallel map collapses to one call of latency · REFINE’s strictly serial chain · the doc-count threshold $D^* = (W - B)/d$ where Stuff overflows and Map-Reduce wins.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: one batch, small enough to count by hand

The module’s premise is that map-reduce is a *control pattern*, not a document trick: you have N independent inputs and want one combined output. There are exactly three shapes the control flow can take, and they trade off the same three currencies — **tokens** (the bill), **latency** (the wall clock), and the **context window** (the cliff you fall off). We pin down all three with one concrete batch.

The workload. A support team has $D = 20$ product reviews to fold into one executive summary. Each review is $d = 800$ tokens. Every model call carries a fixed overhead $B = 500$ tokens (system prompt + instructions). A MAP call distils one review into a $s = 100$ -token structured partial. The model’s context window is $W = 8000$ tokens. Prices and sizes are round so the sums follow on paper; the mechanics are identical at any scale.

Quantity	Symbol	Value here
Documents (reviews) to combine	D	20
Tokens per document	d	800 tokens
Prompt overhead, every call	B	500 tokens
Map summary size (one partial)	s	100 tokens
Context window	W	8,000 tokens
Input price	p_{in}	\$0.50 / 1M tokens
Latency of one model call	L	2 s (illustrative)

Notation. “Input tokens” means the tokens *sent* to the model summed over all calls a strategy makes — that is what you pay for. “Latency” is wall-clock time: calls that run in parallel count once; calls that must run in series add up. “Overflow” means some single call’s input exceeds W and the strategy simply cannot run.

Intuition

Think of the three strategies as three ways to brief one analyst on twenty reviews. STUFF: hand them all twenty at once and ask for the summary — one conversation, but the stack of paper might not fit on the desk. MAP-REDUCE: give twenty junior analysts one review each (all at once), collect their notes, then brief one senior on the notes — two rounds, lots of parallel hands. REFINE: one analyst reads review 1, writes a draft, reads review 2 and revises, ... twenty times in a row — beautifully coherent, but you wait through twenty readings end to end.

2 Step 1 — STUFF: one call, one cliff

STUFF concatenates all D documents into a single prompt and makes *one* call. Its input is the overhead plus the whole payload:

$$I_{\text{stuff}} = \underbrace{B}_{\text{overhead}} + \underbrace{Dd}_{\text{all docs}} . \quad (1)$$

For our batch, $I_{\text{stuff}} = 500 + 20 \cdot 800 = 500 + 16,000 = 16,500$ tokens. The cost is one multiplication, $\text{cost} = p_{\text{in}} I_{\text{stuff}} = 16,500 \times 0.5 \times 10^{-6} = \0.00825 . ✓ The latency is one call, $\approx L = 2$ s. It is the cheapest and fastest strategy — *when it fits*.

But Equation (1) carries a cliff. The call runs only if $I_{\text{stuff}} \leq W$. Here $16,500 > 8,000$, so **Stuff overflows** on this batch: it cannot run at all. The module’s card says it plainly — STUFF “fails when $N \times \text{size} > \text{ctx}$.”

Mini-example — this operation in isolation

The cliff is a line, not a curve. Stuff’s input grows by exactly $d = 800$ tokens per added document, starting from $B = 500$. So solve $B + Dd = W$ for the largest D that still fits: $D^* = (W - B)/d = (8000 - 500)/800 = 9.375$. At $D = 9$ the input is $500 + 7200 = 7,700 \leq 8000$ — fits. At $D = 10$ it is $500 + 8000 = 8,500 > 8000$ — overflows. So **Stuff survives up to 9 documents and dies at 10**, with our numbers. One document tips it over the edge; there is no graceful degradation.

3 Step 2 — MAP-REDUCE: D small calls plus one

MAP-REDUCE never puts all the documents in one prompt. It **maps** each document to a small partial in its own call, then **reduces** the partials in one final call.

Map. D independent calls, each sending overhead plus one document: input $B + d$ per call. Crucially these calls are independent, so they run *in parallel* — their wall-clock cost is one call’s latency, not D .

Reduce. One call that ingests all D partials of s tokens each, plus overhead: input $B + D s$.

Summing the input tokens you actually pay for:

$$I_{\text{mr}} = \underbrace{D(B + d)}_{\text{map, } D \text{ calls}} + \underbrace{(B + D s)}_{\text{reduce, 1 call}} \quad (2)$$

Working the numbers: map is $20 \cdot (500 + 800) = 20 \cdot 1300 = 26,000$; reduce is $500 + 20 \cdot 100 = 2,500$; total $I_{\text{mr}} = 28,500$ tokens, costing $28,500 \times 0.5 \times 10^{-6} = \0.01425 . ✓ Latency is map (parallel $\rightarrow$ one call) plus reduce (one call): $\approx 2L = 4$ s.

The decisive property is in the per-call inputs, not the total. The largest single call here is the reduce at $B + D s = 2,500$ tokens; the map calls are 1,300 each. Both are far under $W = 8000$.

Map-Reduce never overflows on this batch — it pays roughly $1.7\times$ the tokens of Stuff but moves the payload out of any single window.

Intuition

Map-Reduce trades *total* tokens for *per-call* tokens. Stuff sends 16,500 tokens through one window; Map-Reduce sends 28,500 tokens total but never more than 2,500 through any one window. You pay more in aggregate — the overhead B is re-paid on every map call, and the documents still get read — but no individual call ever has to swallow the whole batch. That is the entire reason the pattern scales when Stuff cannot.

4 Step 3 — REFINE: coherent, but strictly serial

REFINE also makes one call per document, but *sequentially*, threading a running draft through the chain. Call 1 reads document 1 and writes draft 1 (input $B + d$). Call i for $i > 1$ reads the running summary ($\approx s$) and document i and revises: input $\approx B + s + d$. The total input tokens are

$$I_{\text{ref}} = (B + d) + (D - 1)(B + s + d). \quad (3)$$

Numerically: $(500 + 800) + 19 \cdot (500 + 100 + 800) = 1300 + 19 \cdot 1400 = 1300 + 26,600 = 27,900$ tokens, costing $\$0.01395$ — essentially the same bill as Map-Reduce. The largest single call is $B + s + d = 1,400$ tokens, comfortably under W , so Refine **never overflows** either.

The catch is not tokens; it is the clock. Every call depends on the draft from the call before it, so they **cannot** run in parallel. Latency is the full chain: $\text{latency}_{\text{ref}} = D \cdot L = 20 \cdot 2 = 40$ s — ten times Map-Reduce’s 4 s and twenty times Stuff’s 2 s.

Watch out

Refine is *structurally identical* to the ReAct loop of Module 0.0: a single state (the draft)

carried forward across D sequential calls. It inherits ReAct’s two liabilities. **Latency** grows linearly in D because nothing parallelises. And **drift** accumulates: each revision can quietly dilute or overwrite an early conclusion, so document 1’s signal may be a faint memory by document 20. Refine buys coherence and ordering at the price of the wall clock and long-chain fidelity — use it only when later inputs genuinely build on earlier ones.

5 Step 4 — the three side by side at $D = 20$

Now put all three in one ledger for the $D = 20$ batch ($B = 500$, $d = 800$, $s = 100$, $W = 8000$, $p_{\text{in}} = \$0.5/1\text{M}$, $L = 2$ s):

Strategy	Calls	Input tokens	Cost	Latency	Overflow?
STUFF	1	16,500	\$0.00825	≈ 2 s	YES ($> W$)
MAP-REDUCE	$D+1 = 21$	28,500	\$0.01425	≈ 4 s	no
REFINE	$D = 20$	27,900	\$0.01395	≈ 40 s	no

Read the row meanings off Equations (1)–(3). Stuff is the cheapest and fastest *but does not run here*. Map-Reduce and Refine cost near-identical tokens; they split on latency (4 s vs 40 s) because map calls parallelise and refine calls do not. Sample check, Map-Reduce input: $20 \cdot 1300 + 2500 = 26,000 + 2,500 = 28,500$. ✓

6 Step 5 — the threshold where Stuff overflows and Map-Reduce wins

Stuff is strictly best *while it fits*: one call, fewest tokens, lowest latency. The whole decision reduces to one question — at how many documents does it stop fitting? Set Stuff’s input equal to the window and solve for D :

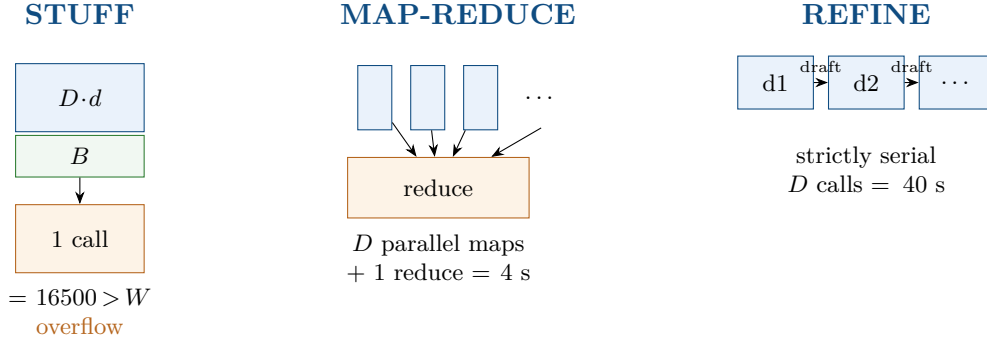
$$B + Dd = W \implies \boxed{D^* = \frac{W - B}{d}} \quad (4)$$

With our numbers $D^* = (8000 - 500)/800 = 9.375$. Since D is an integer, Stuff fits for $D \leq 9$ and overflows for $D \geq 10$. **That crossover is the decision boundary**: below it, use Stuff; at or above it, Stuff is off the table and Map-Reduce (independent items) or Refine (ordered items) takes over. The module’s “fits in context?” question is exactly Equation (4).

D	Stuff input = $B + Dd$	Stuff fits?	Map-Reduce input = $D(B+d)+B+Ds$
5	4,500	yes	7,500
9	7,700	yes (last D that fits)	13,100
10	8,500	no (first overflow)	14,500
20	16,500	no	28,500
50	40,500	no	70,500

Notice Map-Reduce’s input keeps growing too — but it grows as a *total bill*, never as a single-call window. Its largest call is the reduce at $B + Ds$; that overflows only when $D > (W - B)/s = (8000 - 500)/100 = 75$. Past *that* second threshold even Map-Reduce’s reduce step cannot hold all the partials — which is precisely where Topic 2’s hierarchical reduce begins.

7 The three shapes, drawn



Stuff is one tall stack against the window. Map-Reduce is a wide row of tiny parallel calls feeding one reduce — short and fat. Refine is a single thread snaking through every document — narrow and long. The pictures *are* Equations (1)–(3).

8 Strategy cheat-sheet

	Input tokens	Latency
STUFF	$B + Dd$	~ 1 call (L)
MAP-REDUCE	$D(B + d) + (B + Ds)$	~ 2 calls ($2L$): parallel map + reduce
REFINE	$(B + d) + (D-1)(B+s+d)$	$\sim D$ calls (DL): strictly serial
Stuff overflows when	$B + Dd > W \iff D > D^* = (W - B)/d$	
Map-Reduce reduce overflows when	$B + Ds > W \iff D > (W - B)/s$	
Decision (module)	fits? $\rightarrow$ Stuff $\cdot$ independent? $\rightarrow$ Map-Reduce $\cdot$ ordered? $\rightarrow$ Refine	

9 An honest word about these numbers

The sizes here ($B = 500$, $d = 800$, $s = 100$, $W = 8000$, $L = 2$ s) are illustrative round numbers, chosen so the sums follow by hand and so Stuff’s cliff lands at a tidy $D^* = 9.375$. Real values depend on your model and content: a frontier model’s window is 128k–1M tokens, not 8k, which pushes D^* into the hundreds or thousands; verbose reviews make d larger; a richer map schema makes s larger and lowers Map-Reduce’s reduce ceiling. What is *not* illustrative is the structure: Stuff is one call bounded by $B + Dd \leq W$; Map-Reduce pays $D(B + d) + (B + Ds)$ tokens across D parallel maps and one reduce; Refine pays a comparable token bill but over D *serial* calls, so its latency is DL . Change the constants and the dollars and seconds move; the three shapes — one cheap call with a hard cliff, a wide parallel fan-in, and a long serial thread — do not. That trichotomy is the thing to carry forward.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the three-strategy ledger, the overflow threshold, and the scaling table; change D , d , B , s , W , or the price to explore your own workload.

```

1  import math
2
3  # Topic 1 -- Stuff vs Map-Reduce vs Refine, costed by hand.
4  D, d, B, s, W = 20, 800, 500, 100, 8000
5  P_IN = 0.5e-6 # $0.50 / 1M input tokens
6  L = 2.0 # seconds per LLM call (illustrative)
7
8  def stuff(D):
9      inp = B + D * d
10     return inp, P_IN * inp, inp > W, 1 * L
11
12 def mapreduce(D):
13     map_each = B + d # one map call's input
14     reduce_in = B + D * s # the single reduce call's input
15     total = D * (B + d) + reduce_in
16     overflow = (map_each > W) or (reduce_in > W)
17     return map_each, reduce_in, total, P_IN * total, overflow, 2 * L
18
19 def refine(D):
20     each = B + s + d # steady-state per-call input
21     total = (B + d) + (D - 1) * (B + s + d)
22     return each, total, P_IN * total, each > W, D * L
23
24 print("strategy in_tokens cost($) overflow latency(s)")
25 i, c, o, l = stuff(D)
26 print(f"STUFF {i:9d} {c:8.5f} {str(o):8} {l:5.1f}")
27 me, ri, t, c, o, l = mapreduce(D)
28 print(f"MAPREDUCE {t:9d} {c:8.5f} {str(o):8} {l:5.1f}")
29 e, t, c, o, l = refine(D)
30 print(f"REFINE {t:9d} {c:8.5f} {str(o):8} {l:5.1f}")
31
32 print(f"\nMR detail: map_in_each={me}, reduce_in={ri}")
33
34 Dstar = (W - B) / d
35 print(f"\nStuff overflow threshold D* = (W-B)/d = {Dstar}")
36 for Dt in (9, 10):
37     i, _, o, _ = stuff(Dt)
38     print(f" D={Dt:2d}: stuff_in={i}, overflow={o}")
39
40 print("\nScaling (stuff vs map-reduce input tokens):")
41 for Dt in (5, 9, 10, 20, 50):
42     si, _, so, _ = stuff(Dt)
43     _, _, mt, _, mo, _ = mapreduce(Dt)
44     print(f" D={Dt:2d}: stuff={si:6d}({'OVF' if so else 'ok '}) MR={mt:6d}({'OVF' if mo else 'ok '})")
45
46 # Expected output:
47 # strategy in_tokens cost($) overflow latency(s)
48 # STUFF 16500 0.00825 True 2.0
49 # MAPREDUCE 28500 0.01425 False 4.0
50 # REFINE 27900 0.01395 False 40.0
51 #
52 # MR detail: map_in_each=1300, reduce_in=2500
53 #
54 # Stuff overflow threshold D* = (W-B)/d = 9.375
55 # D= 9: stuff_in=7700, overflow=False
56 # D=10: stuff_in=8500, overflow=True

```

```
57 #
58 # Scaling (stuff vs map-reduce input tokens):
59 # D= 5: stuff= 4500(ok ) MR= 7500(ok )
60 # D= 9: stuff= 7700(ok ) MR= 13100(ok )
61 # D=10: stuff= 8500(OVF) MR= 14500(ok )
62 # D=20: stuff= 16500(OVF) MR= 28500(ok )
63 # D=50: stuff= 40500(OVF) MR= 70500(ok )
```

— *end of document* —